push_swap

push_swap est un programme en Go qui calcule une suite d'opérations pour trier une pile d'entiers, en utilisant une seconde pile comme espace temporaire. Le programme affiche les opérations à exécuter, une par ligne. Le projet comprend aussi checker, qui rejoue une suite d'opérations et indique si elle trie correctement la pile.

Règles du jeu

La pile A contient les valeurs initiales et la pile B commence vide. Le sommet de chaque pile est son premier élément. Les seules opérations autorisées sont :

Opération	Effet
sa, sb, ss	Échange les deux premiers éléments de A, de B ou des deux.
pa, pb	Déplace le sommet de B vers A, ou de A vers B.
ra, rb, rr	Place le sommet de A, de B ou des deux en bas de sa pile.
rra, rrb, rrr	Place le dernier élément de A, de B ou des deux au sommet.

Le résultat est correct lorsque A est triée par ordre croissant et que B est vide. Les entrées invalides et les valeurs dupliquées sont rejetées.

Organisation

```
.
■■■ checker/
■   ■■■ main.go                 # Point d'entrée du checker
■   ■■■ logic/                  # Validation et rejeu des opérations
■■■ entiers/                    # Analyse et validation des arguments entiers
■■■ instructions/
■   ■■■ echanger/               # sa, sb, ss
■   ■■■ pousser/                # pa, pb
■   ■■■ rotation_haut/          # ra, rb, rr
■   ■■■ rotation_bas/           # rra, rrb, rrr
■■■ piles/                      # État global partagé des piles A et B
■■■ push-swap/
■   ■■■ main.go                 # Point d'entrée du solveur
■   ■■■ logic/
■       ■■■ pushswap.go         # Sélection de l'algorithme
■       ■■■ petits_tailles.go   # Tri spécialisé jusqu'à sept éléments
■       ■■■ radix.go            # Tri radix par rang pour les entrées plus grandes
■■■ go.mod
■■■ README.md
```

Les instructions modifient l'état partagé dans piles.A et piles.B. Les algorithmes consultent ce même état après chaque opération, afin de ne pas prendre de décision à partir d'une copie périmée des piles.

Algorithmes

Deux éléments : Échange si nécessaire.

Trois éléments : Trie avec des échanges et rotations.

Quatre à sept éléments : Pousse les minimums vers B jusqu'à ce qu'il reste trois éléments dans A, trie ces trois éléments, puis replace les éléments dans A.

Plus de sept éléments : Compresse les valeurs en rangs croissants, puis effectue un tri radix binaire. La compression permet de gérer les entiers négatifs et les valeurs de toute taille sans supposer une plage fixe.

Utilisation

Le projet n'utilise que la bibliothèque standard de Go.

Afficher les opérations de tri :

```
go run ./push-swap 3 2 1
```

Les arguments peuvent aussi être fournis dans une seule chaîne :

```
go run ./push-swap "3 2 1"
```

Vérifier la suite d'opérations :

```
go run ./push-swap 3 2 1 | go run ./checker 3 2 1
```

Le checker affiche OK si A est triée et B vide, et KO sinon.

Construire les exécutables :

```
go build -o push-swap.exe ./push-swap
go build -o checker.exe ./checker
```

Tests

Lancer tous les tests :

```
go test ./...
```

Les tests vérifient les opérations de base, le parsing des arguments, le checker et le résultat du solveur sur plusieurs cas représentatifs, ainsi que le rejet des arguments ou instructions mal formatés.